Cornell Bowers CIS
Computer Science

Applied High-Performance and Parallel Computing

SpMV + OpenMP (Vol. 1)

Professor Giulia Guidi

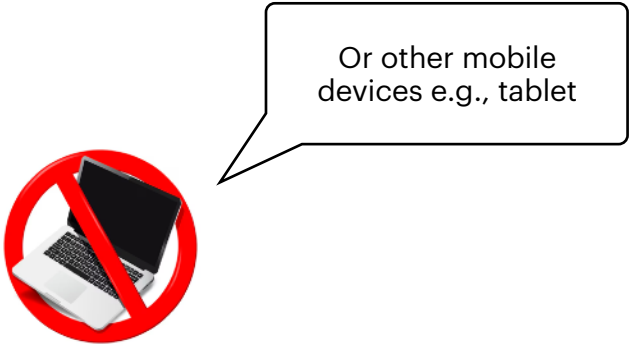
02/10/2026

A number of slides were taken from or inspired by slides by Aydin Buluç, David Bindel, and Tim Mattson

Plan for Today

- SpMV
- Shared-Memory (Vol. 1)

no smartphones



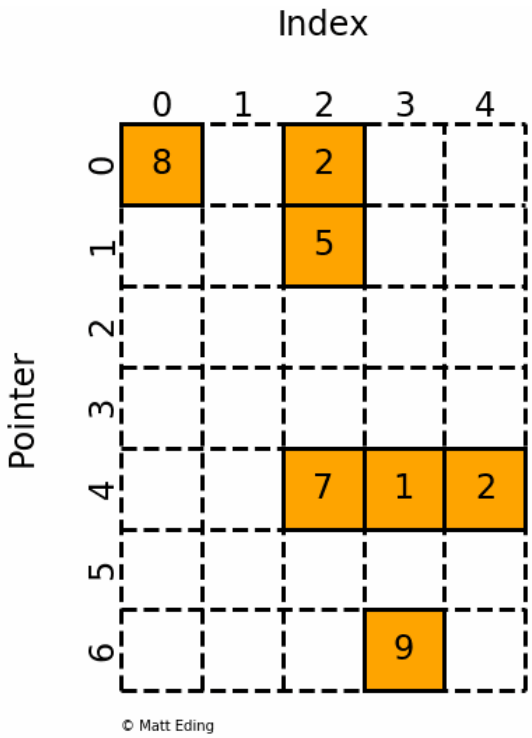
no laptop

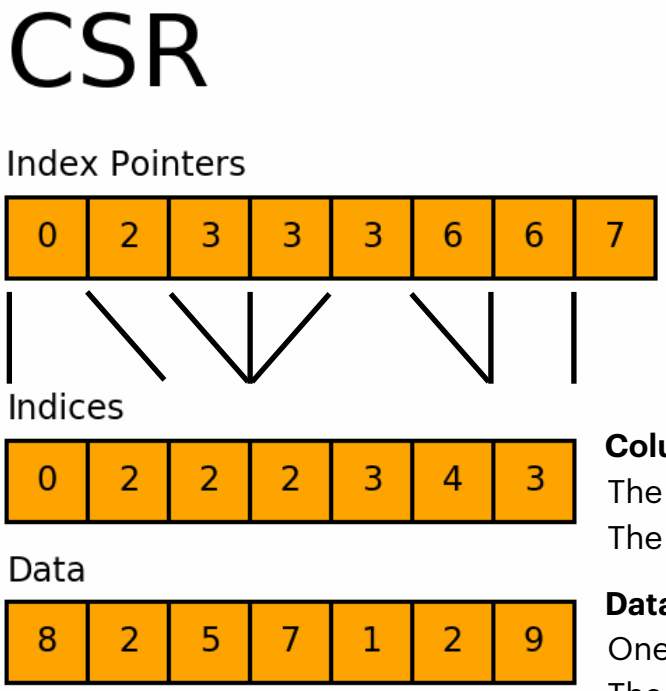
Why? Research shows that they slow us down in a learning environment.
I recommend **paper and pen** for taking notes

Credit to: https://users.cs.utah.edu/~pandey/courses/cs6530/fall23/index.html

Sparse Matrix Vector Multiply (SpMV)

Compressed Sparse Row (CSR)





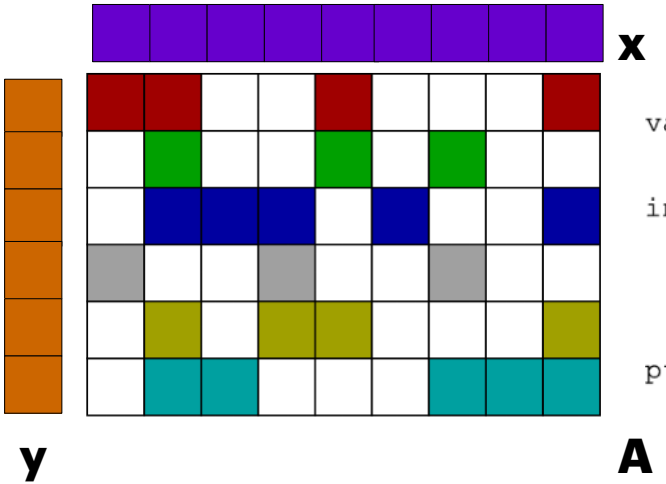
Row pointer
The **index** is the row id
The **values** are [start:stop] in the column indices array

Column indices
The **length** is the number of nonzeros
The **values** are column indices

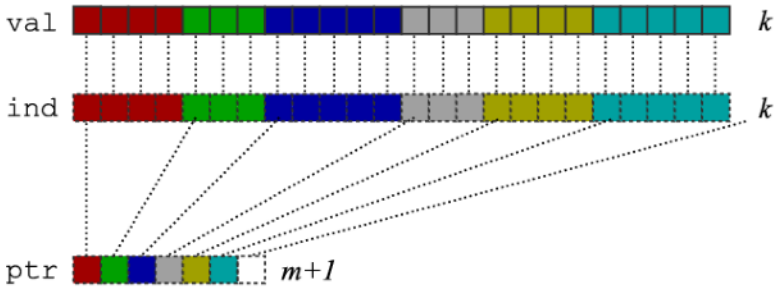
Data values
One-to-one correspondence with column
The **values** are the nonzeros

SpMV

SpMV kernel: y(i) = y(i) + A(i, j)·x(j)



Representation of **A**



for each row **i**

for **k** = **ptr[i]** to **ptr[i+1]-1** do

y[i] = **y[i]** + **val[k]** * **x[ind[k]]**

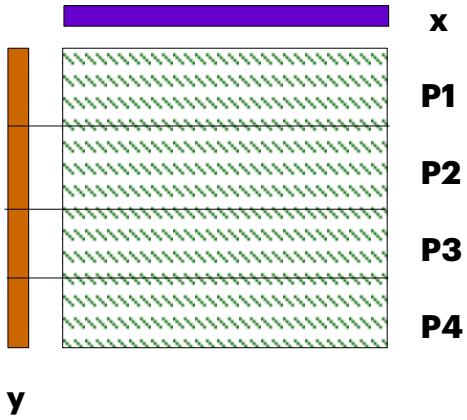
Parallel SpMV

SpMV kernel: $y(i) = y(i) + A(i, j) \cdot x(j)$

Questions:

- On which processors we store $y(i)$, $x(i)$, and $A(i, j)$?
- On which processors we compute $y(i)$= sum (from 1 to n) $A(i, j) * x(j)$ = (row i of A) * x?

A sparse dot product!



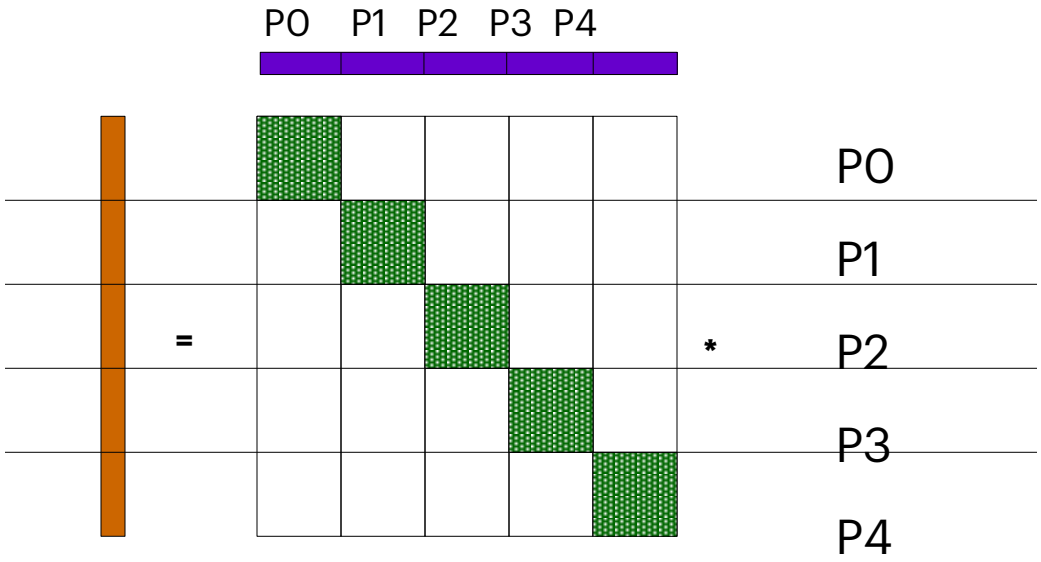
Partitioning:

- Partition index set $\{1, \ldots, n\} = N_1 \cup N_2 \cup \ldots \cup N_p$
- For all i in N_k, processor k stores y[i], x[i], and row i of A
- For all i in N_k, processor k computes y[i] = (row i of A) * x

"Owner computes" rule: processor k computes the y[i]s it owns!

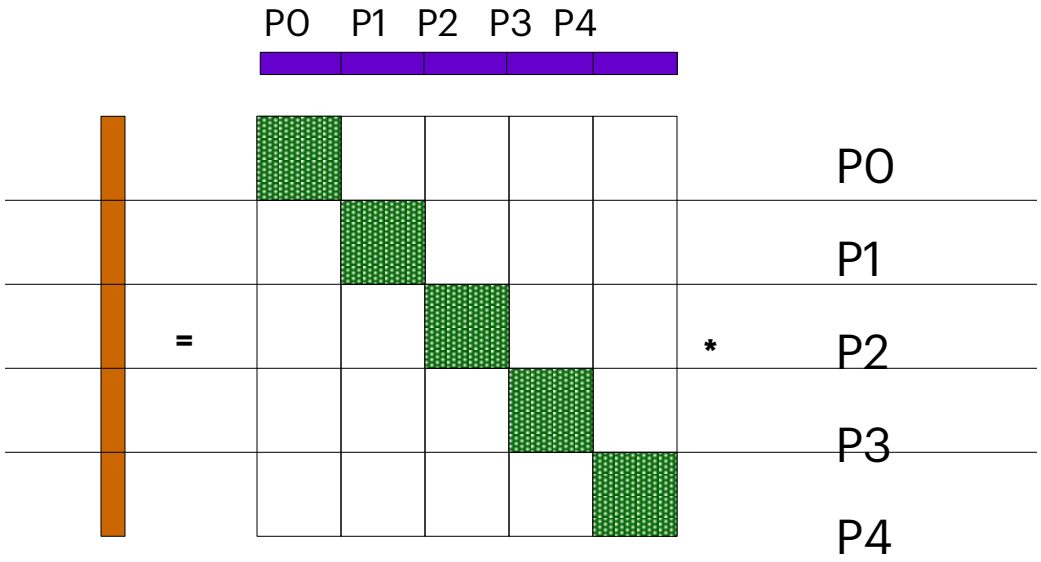
Sparse Matrix Reordering

- The ideal matrix structure for parallelism is **block diagonal**



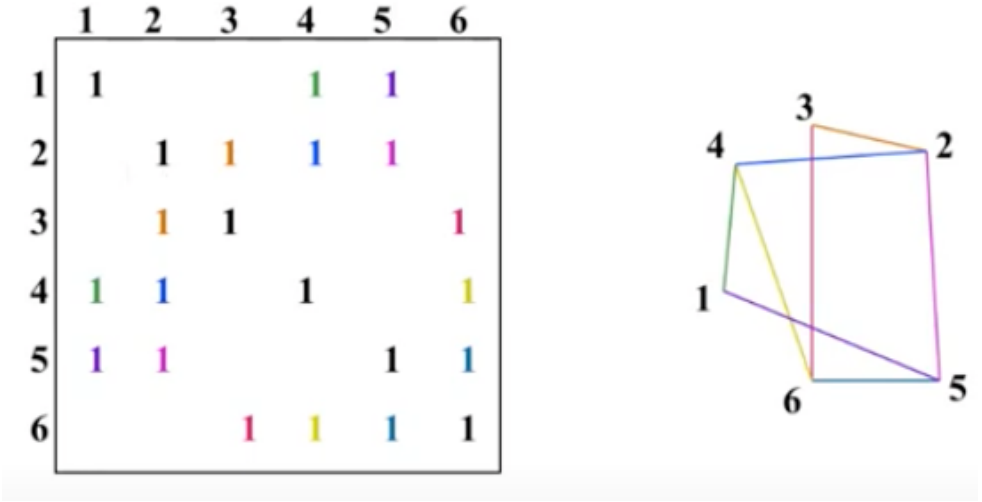
Sparse Matrix Reordering

- The ideal matrix structure for parallelism is **block diagonal**
- Can we reorder the rows/columns to get close to this?



Graph Partitioning

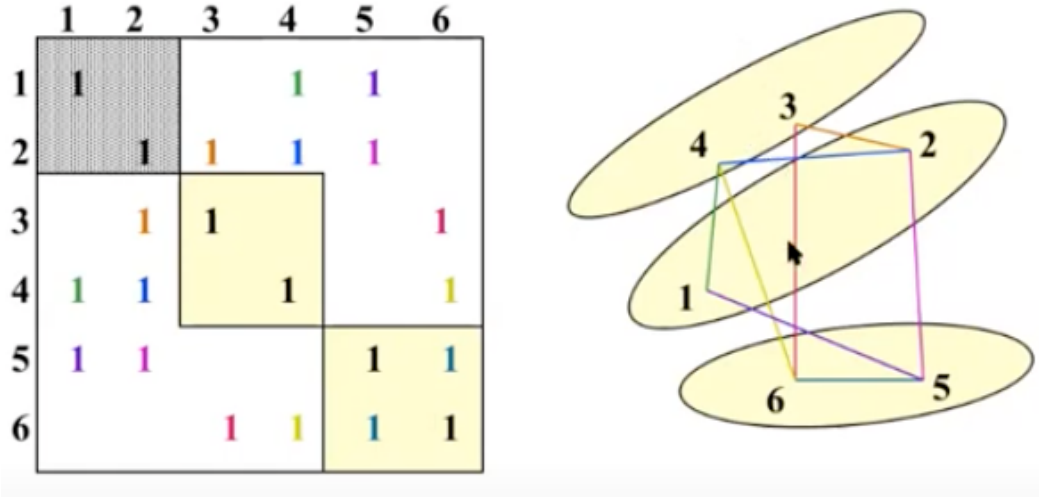
Relationship between matrix and graph:



- A edge in the graph is a nonzero in the matrix
- If divided over 3 processors, there are 14 nonzeros outside the diagonal blocks, which are the 7 (bidirectional) edges

Graph Partitioning

Relationship between matrix and graph:



- A edge in the graph is a nonzero in the matrix
- If divided over 3 processors, there are 14 nonzeros outside the diagonal blocks, which are the 7 (bidirectional) edges

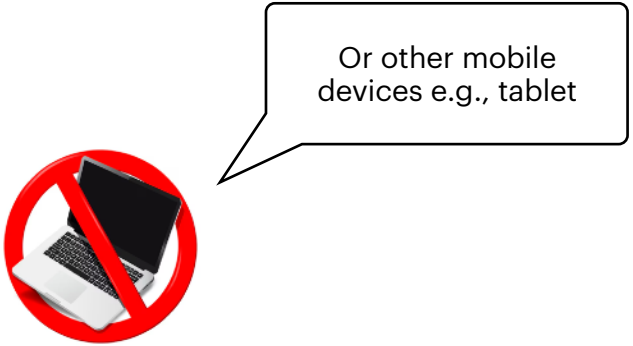
Common Problems Summary

- Load balancing:
  - Done **statically**, using graph partitioning
    - Discrete event simulation
    - SpMV
  - Done **dynamically** if workload changes significantly at runtime
- Linear algebra:
  - Require to solve linear systems (sparse and dense)
  - Eigenvalues problems use similar techniques
- Fast particle methodologies:
  - $O(n \log n)$ instead of $O(n^2)$

Plan for Today

- SpMV
- Shared-Memory (Vol. 1)

no smartphones



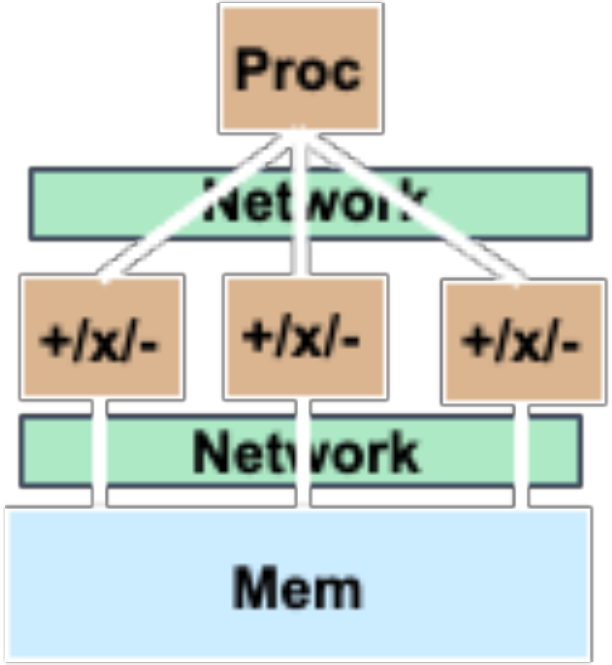
no laptop

Why? Research shows that they slow us down in a learning environment.
I recommend **paper and pen** for taking notes

Credit to: https://users.cs.utah.edu/~pandey/courses/cs6530/fall23/index.html

Single Core Machine

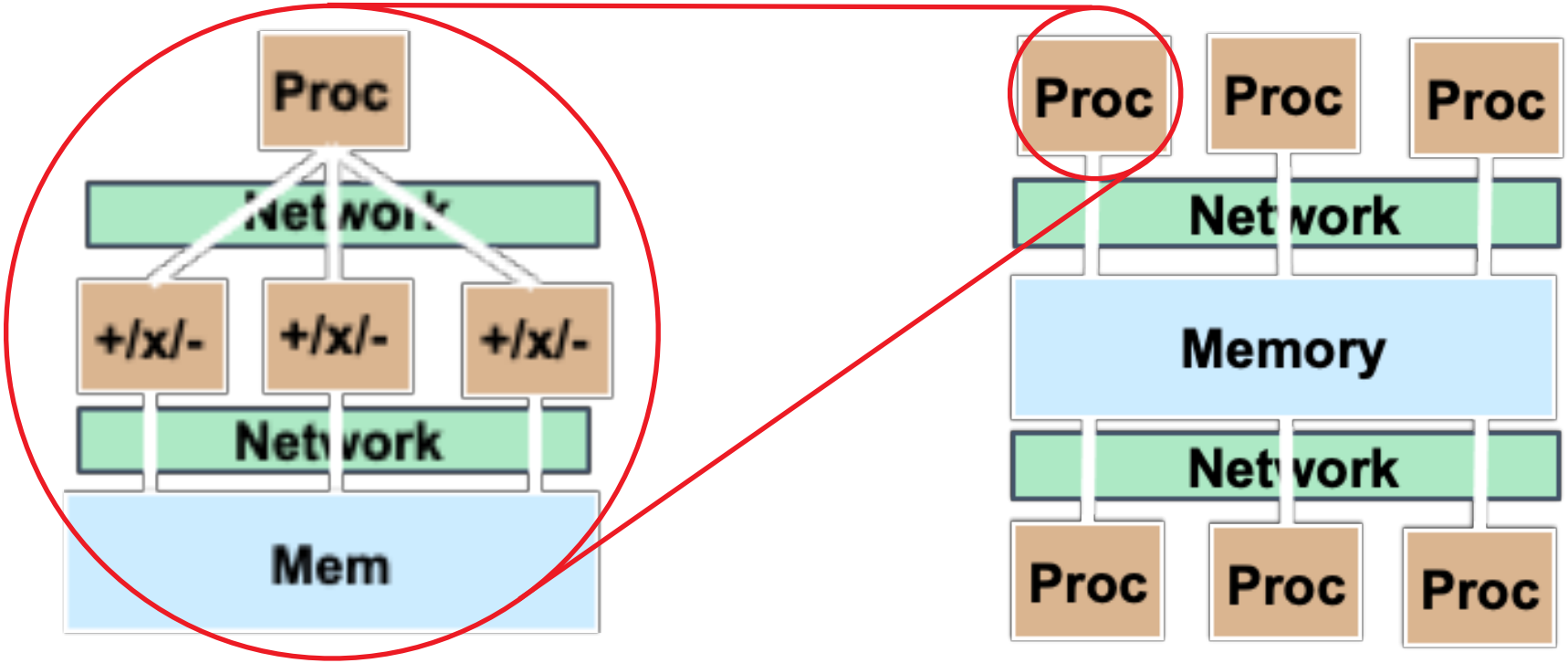
Summary



- One processor with:
  - pipeline,
  - multiple ALUs,
  - multiple instructions going on at once (out-of-order execution)
- Registers, caches, and main memory (atomic access)

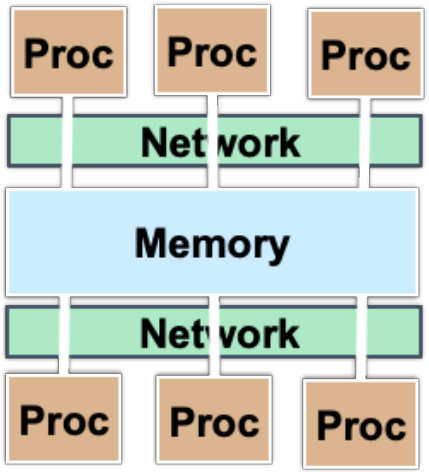
Shared Memory Machine

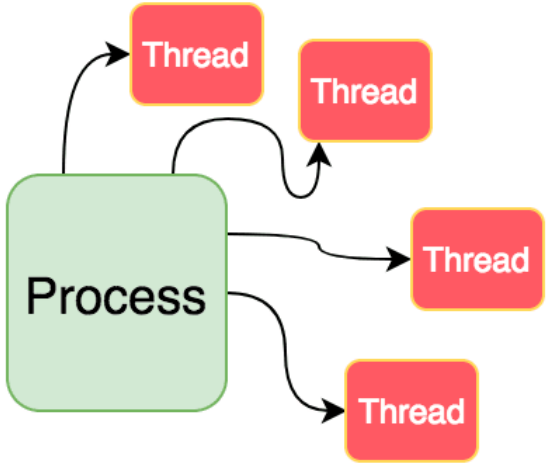
Basically, multiple processors in a box



What's a Thread?

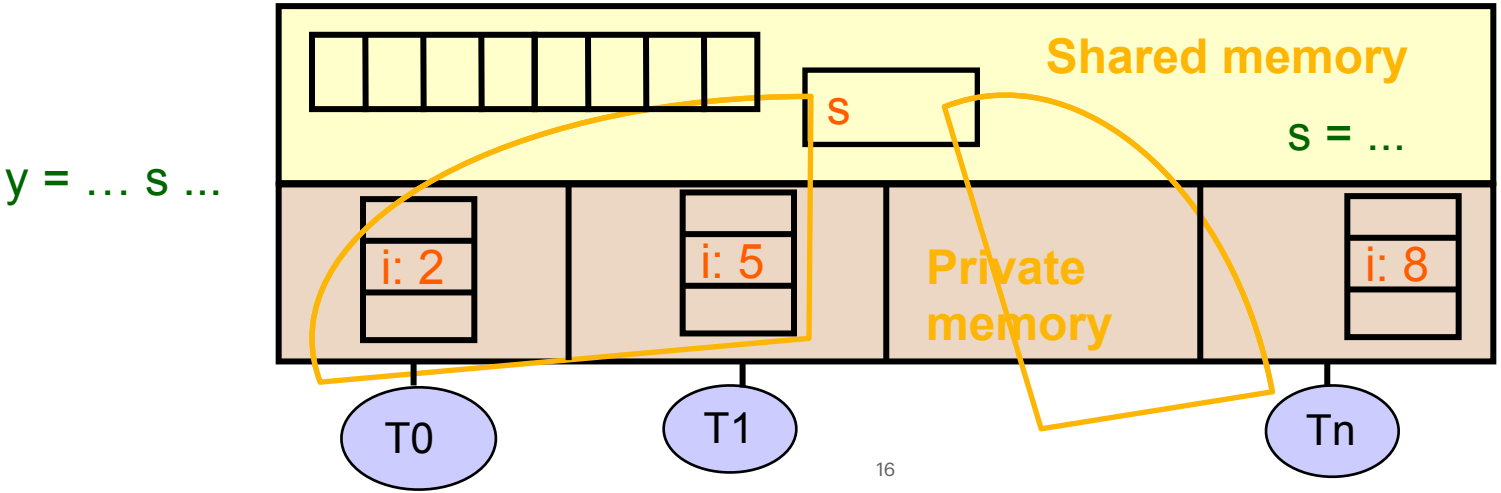
- A thread is a **sequential execution flow within a process** (individual stack and program counter)
- Threads **transparently share the memory address space** and other process resources
- Threads in the process **execute concurrently**, *possibly on different cores* over time





Shared Memory Programming Model

- Program is a collection of threads of control
  - Can be created dynamically, mid-execution, in some languages
- Each thread has a set of **private variable**, e.g., local stack variables
- And, a set of **shared variable**, e.g., static variables, shared common blocks, or global heap



Shared Memory Programming Model

- Program is a collection of threads of control
  - Can be created dynamically, mid-execution, in some languages
- Each thread has a set of **private variable**, e.g., local stack variables
- And, a set of **shared variable**, e.g., static variables, shared common blocks, or global heap
  - Threads communicate **implicitly** by writing and reading shared variables
  - Threads coordinate by **synchronizing** on shared variables

Shared Memory Programming Model

- Program is a collection of threads of control
  - Can be created dynamically, mid-execution, in some languages
- Each thread has a set of **private variable**, e.g., local stack variables
- And, a set of **shared variable**, e.g., static variables, shared common blocks, or global heap
  - Threads communicate **implicitly** by writing and reading shared variables
  - Threads coordinate by **synchronizing** on shared variables

Challenges:

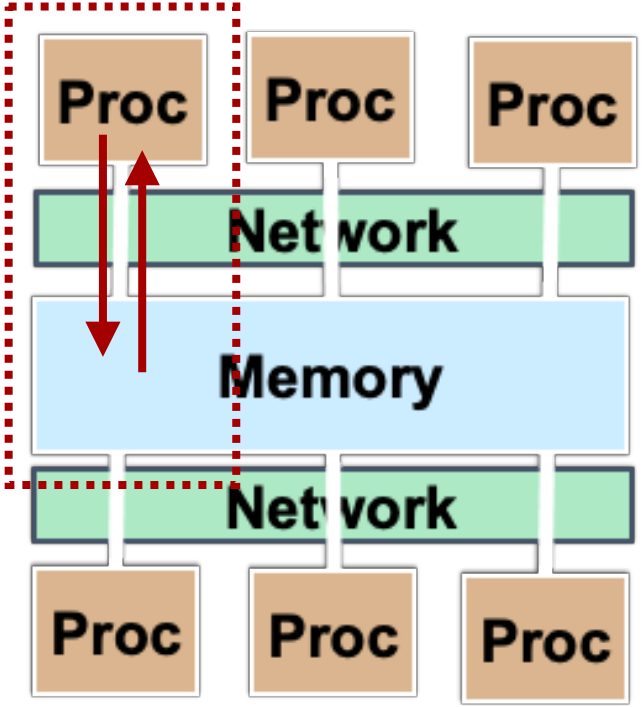
- Synchronization **costs** cycles even with no contention
- **Contention** reduces parallelism
- **Cache coherency** can slow down even non-contending access

Shared Memory Machine

The memory model

The model of a **single** processor:

- Return **last write**



Shared Memory Machine

The memory model

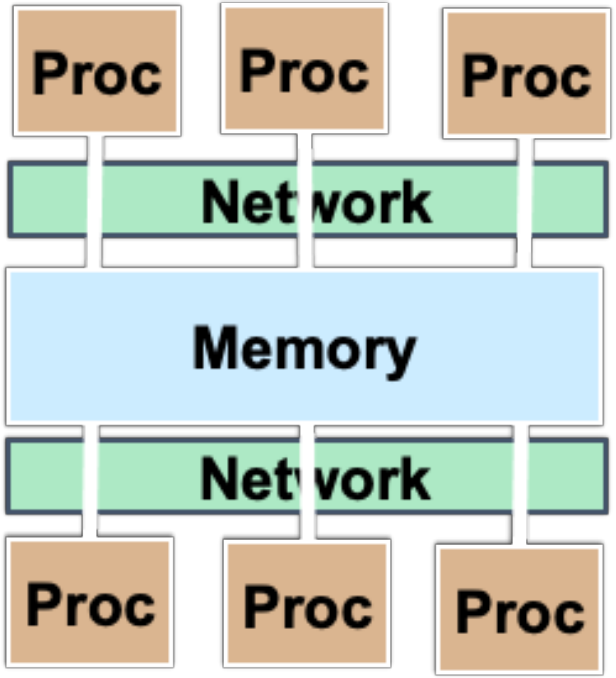
The model of a **single** processor:

- Return **last write**

But now we need to think about **multiple** processors!

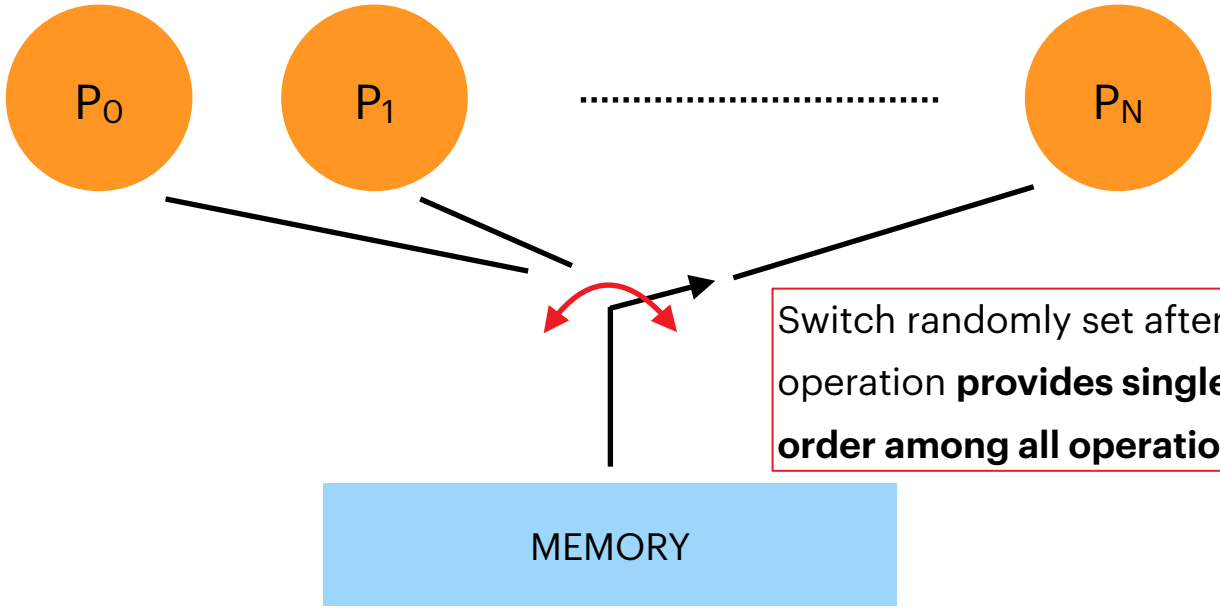
The simplest *generalization* is **sequential consistency**:

- Each process runs in program order
- Instructions from different processes are interleaved



Sequential Consistency

Processors issues memory operations **in program order**



Switch randomly set after each memory operation **provides single sequential order among all operations**

Sequential Consistency

The good, the bad, and the ugly

The **good**: the program behavior is “intuitive”

- Processes don’t see garbage values
- Time always moves forward

The **bad**: *cache coherence*

- Coherence: different copies, same value
- Requires nontrivial hardware support

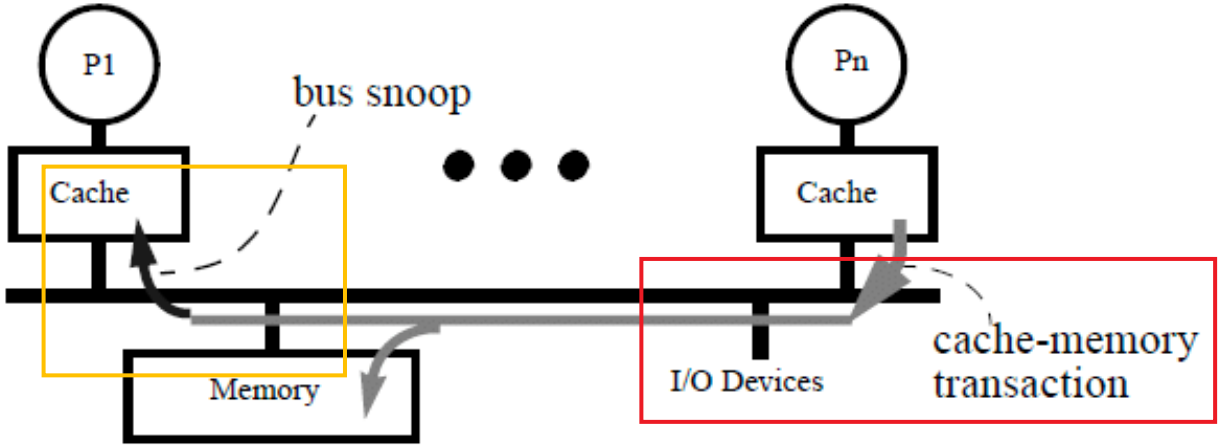
The **ugly**: also an issue for optimizing compiler!

Cheaper ***relaxed*** consistency models exist

Snoopy Bus Protocol

Basic idea:

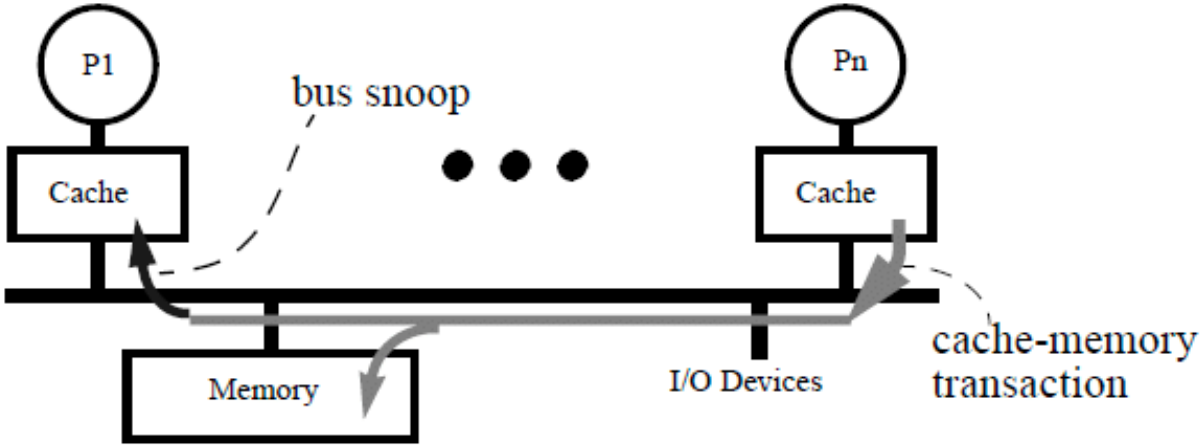
- Caches **broadcast** memory operations on memory bus
- Cache controllers **"snoop"** on all bus transactions



Snoopy Bus Protocol

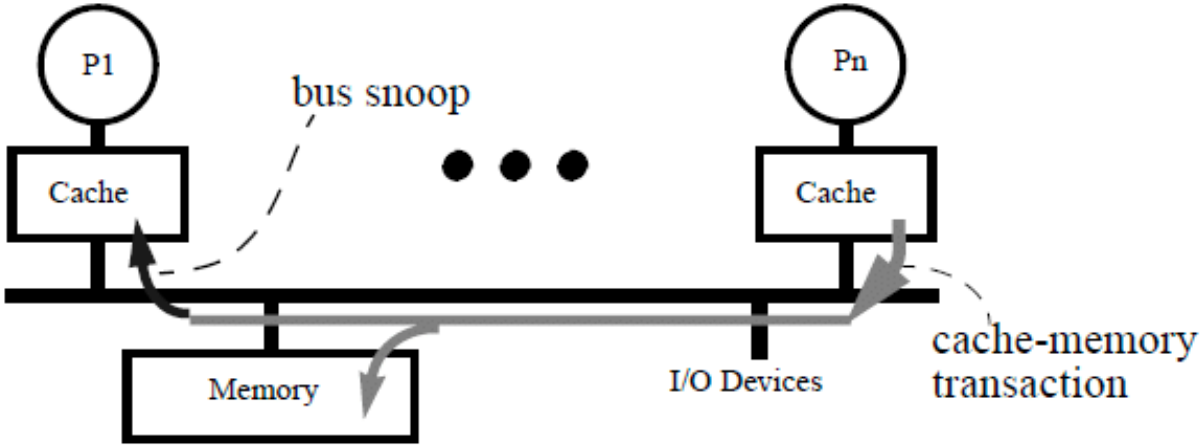
Challenges:

- Bus **bandwidth limits scaling**
- Contending write are **slow**



Snoopy Bus Protocol

Other protocols exist (e.g., directory based) but usually give up on ***full*** sequential consistency



True Sharing

True sharing:

- It happens when **frequent writes** cause a bottleneck
- E.g., two cores that try to access and modify the same word, resulting in continuous invalidation of the cache line in the other core

Reducing true sharing:

- Use of independent copies of the data (if possible)

False Sharing

False sharing:

- Distinct variables on same cache block
- E.g., two cores try to access and modify the two different words within the same cache line, resulting in continuous invalidation of the cache line in the other core

Reducing false sharing:

- Use private or *threadprivate* data as much as possible
- Use the compiler’s optimization features to eliminate memory load/stores
- Pad data structures so that each thread's data resides on a different cache line
- Change or modify data structures so there is less sharing of data in the thread pool

OpenMP = Open specification for Multi-Processing

A standard API for multi-threaded code:

- Only one specification — multiple implementations
- Lightweight syntax
- C, C++, or Fortran (with appropriate compiler support)

It's high level:

- Preprocessor/compiler directives (80%)
- Library calls (19%)
- Environment variables (1%)

Basic syntax: `#omp` *construct* [*clause* ...]

- It usually affects structured blocks (one way in/out)
- OK to have `exit()` in such a block

A Programmer's View of OpenMP

OpenMP is a portable, threaded, shared-memory programming specification with light syntax:

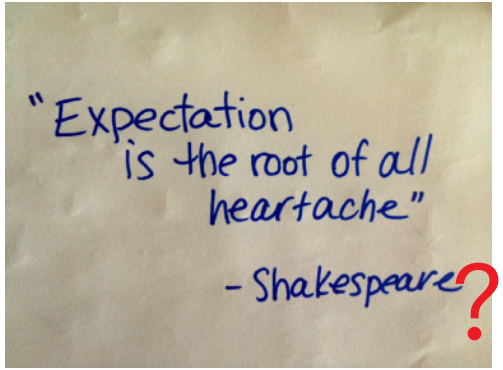
- Requires compiler support (C, C++, Fortran)

OpenMP **will**:

- Enable separation of a program into serial regions and parallel regions,
- Hide stack management
- Provide synchronization constructs

OpenMP **will not**:

- Parallelize automatically
- Guarantee speedup
- Provide freedom from data races



Source: https://evenifministries.wordpress.com/2019/11/27/expectations/

Parallel Hello World in OpenMP

```
#include <stdio.h>


int main()
{

 printf(“ hello ”);
 printf(“ world \n”);

return 0;
}
```

Parallel Hello World in OpenMP

```
1 #include <stdio.h>
2 #include <omp.h>
3
4 int main()
5 {
6  #pragma omp parallel
7  {
8    printf(“ hello ”);
9    printf(“ world \n”);
10 }
11 return 0;
12 }
```

GCC/G++ compiler has OpenMP support by default:

- `gcc -c -fopenmp foo.c`
- `gcc -o -fopenmp mycode.x foo.c`

Parallel Hello World in OpenMP

```
#include <stdio.h>
#include <omp.h>

int main()
{
 #pragma omp parallel
 {
   printf(“ hello ”);
   printf(“ world \n”);
 }
return 0;
}
```

OpenMP include file

Parallel region with default number of threads

End of parallel region

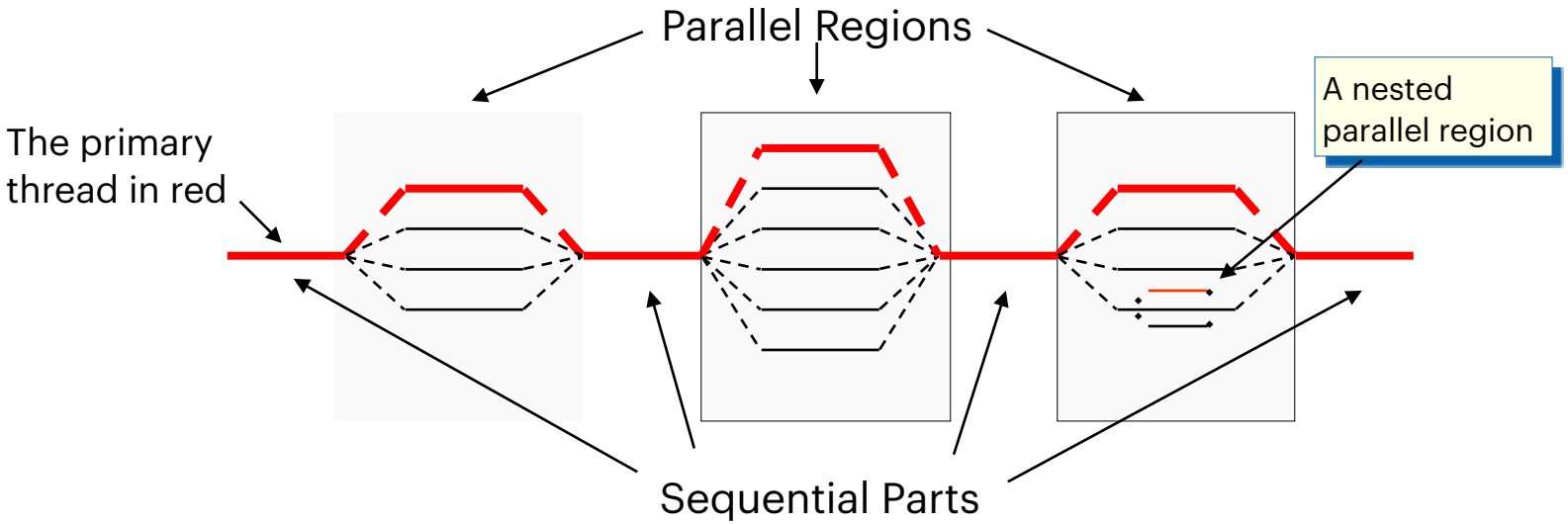
Sample output:

```
hello hello world
world
hello hello world
world
```

- The statements are interleaved based on how the operating system schedules the threads
- The code is **not** enforcing any synchronization

Fork-Join Parallelism

- A master (primary) thread spawns a team of threads as needed
- Parallelism added incrementally until performance objectives are met, i.e., the sequential program evolves into a parallel program



Thread Creation

- You create threads in OpenMP with the parallel construct
- E.g., to create a 4 thread parallel region:

```
double A[1000];
omp_set_num_threads(4);
#pragma omp parallel
{
        int ID = omp_get_thread_num();
        pooh(ID,A);
}
```

Runtime function to request a certain number of threads

Each thread executes a copy of the code within the structured block

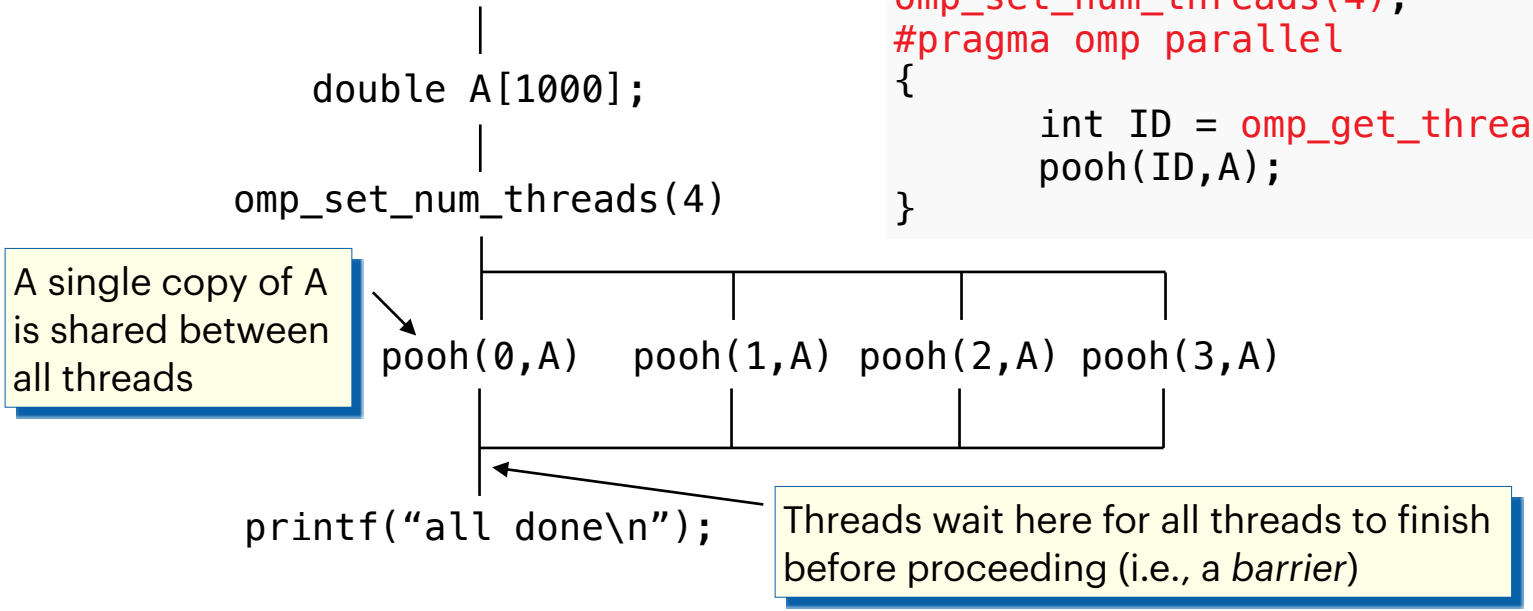
Runtime function returning a thread ID

- Each thread calls `pooh(ID,A)` for ID = 0 to 3

Thread Creation

- Each thread executes the same code **redundantly**

```
double A[1000];
omp_set_num_threads(4);
#pragma omp parallel
{
        int ID = omp_get_thread_num();
        pooh(ID,A);
}
```



Thread Creation

- You create threads in OpenMP with the parallel construct
- You can request a number of threads with `omp_set_num_threads()`
- But is the number of threads requested the number you actually get? NO!
  - Once a team of threads is established, the system will not reduce the size of the team

```
double A[1000];
omp_set_num_threads(4);
#pragma omp parallel
{
    int ID = omp_get_thread_num();
    int nthreads = omp_get_num_threads();
    pooh(ID,A);
}
```

Runtime function to request a certain number of threads

Each thread executes a copy of the code within the structured block

Runtime function to return actual number of threads in the team

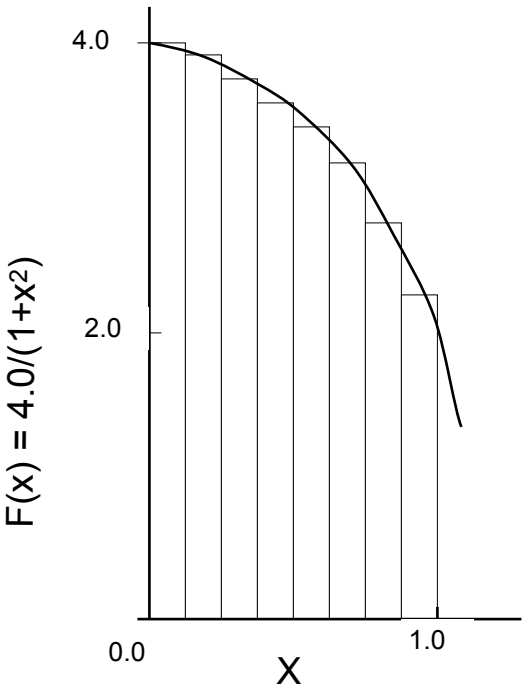
Thread Creation

Control number of threads:

- Default: the system chooses (= number cores?)
- Environment: `export OMP_NUM_THREADS=4`
- Function call: `omp_set_num_threads(4)`
- Clause: `#pragma omp parallel num_threads(4)`

Example

Numerical Integration



In math, we know that:

$$\int_0^1 \frac{4.0}{(1+x^2)}\, dx = \pi$$

Then, we can approximate the integral as a sum of rectangles:

$$\sum_{i=0}^{N} F(x_i)\Delta x \approx \pi$$

Each rectangle has width Δx and height $F(x_i)$ at the middle of interval i

Serial PI Program

```
static long num_steps = 100000;
double step;
int main ()
{
    int i;
    double x, pi, sum = 0.0;

    step = 1.0/(double) num_steps;

    for (i = 0; i < num_steps; i++)
    {
          x = (i + 0.5) * step;
          sum = sum + 4.0/(1.0 + x * x);
    }
    pi = step * sum;
}
```

Serial PI Program

```
#include <omp.h>
static long num_steps = 100000;
double step;
int main ()
{
    int i;
    double x, pi, sum = 0.0;
    double tdata;

    step = 1.0/(double) num_steps;
    tdata = omp_get_wtime();
    for (i = 0; i < num_steps; i++)
    {
          x = (i + 0.5) * step;
          sum = sum + 4.0/(1.0 + x * x);
    }
    pi = step * sum;
    tdata = omp_get_wtime() - tdata;
    printf(" pi = %f in %f secs\n",pi, tdata);
}
```

The library routine `get_omp_wtime()` is used to find the elapsed "wall time" for blocks of code

Parallel PI Program

```
#include <omp.h>
static long num_steps = 100000;
double step;
#define NUM_THREADS 4
void main ()
{
    int i, nthreads;
    double pi, sum[NUM_THREADS];
    step = 1.0/(double) num_steps;
    omp_set_num_threads(NUM_THREADS);
    #pragma omp parallel
    {
        int i, id, nthrds;
        double x;
        id = omp_get_thread_num();
        nthrds = omp_get_num_threads()
        if (id == 0)
            nthreads = nthrds;
        for (i = id, sum[id] = 0.0; i < num_steps; i = i+nthrds)
        {
            x = (i + 0.5)*step;
            sum[id] += 4.0/(1.0 + x * x);
         }
    }
    for(i = 0, pi = 0.0; i < nthreads; i++)
        pi += sum[i] * step;
}
```

Original serial program with 100 million steps ran in **1.83 seconds** using **Intel** compiler on an Intel Core i5 (**2013**)

Threads	Parallel Code
1	1.86
2	1.03
3	1.08
4	0.97

Using 4 threads, not even 2x!

Parallel PI Program

```
#include <omp.h>
static long num_steps = 100000;
double step;
#define NUM_THREADS 8
void main ()
{
    int i, nthreads;
    double pi, sum[NUM_THREADS];
    step = 1.0/(double) num_steps;
    omp_set_num_threads(NUM_THREADS);
    #pragma omp parallel
    {
        int i, id, nthrds;
        double x;
        id = omp_get_thread_num();
        nthrds = omp_get_num_threads()
        if (id == 0)
            nthreads = nthrds;
        for (i = id, sum[id] = 0.0; i < num_steps; i = i+nthrds)
        {
            x = (i + 0.5)*step;
            sum[id] += 4.0/(1.0 + x * x);
         }
    }
    for(i = 0, pi = 0.0; i < nthreads; i++)
        pi += sum[i] * step;
}
```

Original serial program with 1 billion steps ran in **0.99 seconds** using **GCC** compiler on an Intel Core i7 (**2020**)

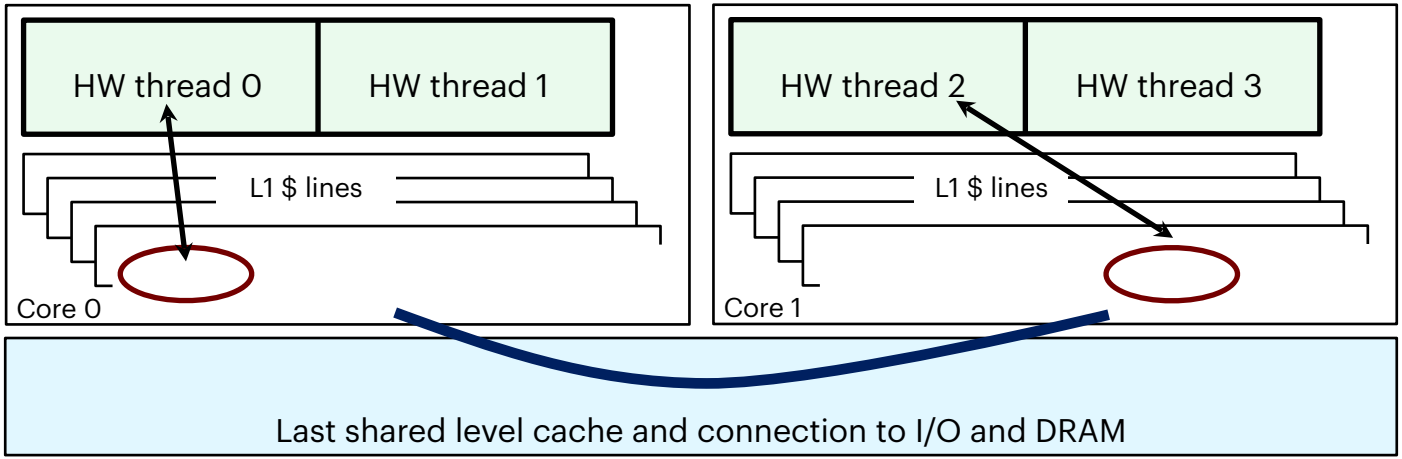
Threads	Parallel Code
1	1.102
2	0.512
4	0.280
8	0.276

Using 8 threads, 4x, not bad, not ideal!

Any idea why this might be happening?

False Sharing!

- Distinct variables on same cache block
- E.g., two cores try to access and modify the two different words within the same cache line, resulting in continuous invalidation of the cache line in the other core



- In out parallel PI program, the array sum is allocated as a contiguous chuck of memory

Parallel PI Program

```
#include <omp.h>
static long num_steps = 100000;
double step;
#define NUM_THREADS 8
#define PAD 8
void main ()
{
    int i, nthreads;
    double pi, sum[NUM_THREADS][PAD];
    step = 1.0/(double) num_steps;
    omp_set_num_threads(NUM_THREADS);
    #pragma omp parallel
    {
        int i, id, nthrds;
        double x;
        id = omp_get_thread_num();
        nthrds = omp_get_num_threads()
        if (id == 0)
            nthreads = nthrds;
        for (i = id, sum[id] = 0.0; i < num_steps; i = i+nthrds)
        {
            x = (i + 0.5)*step;
            sum[id][0] += 4.0/(1.0 + x * x);
         }
    }
    for(i = 0, pi = 0.0; i < nthreads; i++)
        pi += sum[i][0] * step;
}
```

I'm assuming 64 byte L1 cache line size

Pad the array so each sum value is in a different cache line

Parallel Padded PI Program (2013)

```
#include <omp.h>
static long num_steps = 100000;
double step;
#define NUM_THREADS 8
#define PAD 8
void main ()
{
    int i, nthreads;
    double pi, sum[NUM_THREADS][PAD];
    step = 1.0/(double) num_steps;
    omp_set_num_threads(NUM_THREADS);
    #pragma omp parallel
    {
        int i, id, nthrds;
        double x;
        id = omp_get_thread_num();
        nthrds = omp_get_num_threads()
        if (id == 0)
            nthreads = nthrds;
        for (i = id, sum[id] = 0.0; i < num_steps; i = i+nthrds)
        {
            x = (i + 0.5)*step;
            sum[id][0] += 4.0/(1.0 + x * x);
         }
    }
    for(i = 0, pi = 0.0; i < nthreads; i++)
        pi += sum[i][0] * step;
}
```

Original serial program with 100 million steps ran in **1.83 seconds** using **Intel** compiler on an Intel Core i5 (**2013**)

Threads	Parallel Code	Parallel Pad Code
1	1.86	1.86
2	1.03	1.01
3	1.08	0.69
4	0.97	0.53

3x

False Sharing

Reducing false sharing:

- Pad data structures so that each thread's data resides on a different cache line

sum1 and sum2 are in the same cache line:

```
int sum1;
int sum2;

void thread1(int v[], int v_count) {
    sum1 = 0;
    for (int i = 0; i < v_count; i++)
        sum1 += v[i];
}

void thread2(int v[], int v_count) {
    sum2 = 0;
    for (int i = 0; i < v_count; i++)
        sum2 += v[i];
}
```

sum1 and sum2 are on different cache lines:

```
int __attribute__((aligned(64))) sum1;
int __attribute__((aligned(64))) sum2;

void thread1(int v[], int v_count) {
    sum1 = 0;
    for (int i = 0; i < v_count; i++)
        sum1 += v[i];
}

void thread2(int v[], int v_count) {
    sum2 = 0;
    for (int i = 0; i < v_count; i++)
        sum2 += v[i];
}
```

Useful Self-Explanatory Slides

These slides were not (intended to be) covered in class

To review at home!

Performance Tips

- You should experiment to find the best number of threads on your system
- Put as much code as possible inside parallel region(s)
  - Amdahl's law: If 1/s of the program is sequential, then you cannot ever get a speedup better than s
  - So if 1% of a program is serial, speedup is limited to 100, no matter how many processors it is computed on
- Try to have large parallel region(s)
  - To minimize overheads, e.g., starting and stopping threads, executing barrier, moving data into cache
  - Directives can be "orphaned"; procedure calls inside regions are fine
- Runtime routines are your friend
  - Usually very efficient and allow maximum control over thread behavior
- Barriers are expensive
  - Especially with large numbers of threads, they can be slow

To review at home!

Snoopy Bus Protocol

Challenges:

- Bus **bandwidth limits scaling**
- Contending write are **slow**

Other protocols exist (e.g., directory based) but usually give up on ***full*** sequential consistency:

- It avoids broadcasting request to all processes on a miss
- It maintains a directory of which processes have cached copies of the block
- On a miss, send message to directory
- Directory determines what (if any) protocol action is required (e.g., invalidation)
- Directory waits for protocol action to finish, then responds to original request

To review at home!

The OpenMP Common Core

OpenMP pragma, function, or clause	Concepts
#pragma omp parallel	Parallel region, teams of threads, structured block, interleaved execution across threads
int omp_get_thread_num() int omp_get_num_threads()	Create threads with a parallel region and split up the work using the number of threads and thread ID
double omp_get_wtime()	Speedup and Amdahl's law. False Sharing and other performance issues
setenv OMP_NUM_THREADS N	Internal control variables. Setting the default number of threads with an environment variable
#pragma omp barrier #pragma omp critical	Synchronization and race conditions. Revisit interleaved execution.
#pragma omp for #pragma omp parallel for	Worksharing, parallel loops, loop carried dependencies
reduction(op:list)	Reductions of values across a team of threads
schedule(dynamic [,chunk]) schedule (static [,chunk])	Loop schedules, loop overheads and load balance
private(list), firstprivate(list), shared(list)	Data environment
nowait	Disabling implied barriers on workshare constructs, the high cost of barriers, and the flush concept (but not the flush directive)
#pragma omp single	Workshare with a single thread
#pragma omp task #pragma omp taskwait	Tasks including the data environment for tasks.